

Modül 6: Pencere Fonksiyonları (Window Functions)

Veri biliminde en büyük ihtiyaçlardan biri, satırları gruplandırırken her bir satırın detayını kaybetmemektir. GROUP BY veriyi daraltıp özetlerken, **Pencere Fonksiyonları** her satır için bir hesaplama yapar ama tablo yapısını bozmaz.

6.1. Temel Mantık: OVER() ve PARTITION BY

Pencere fonksiyonlarını kullanmak için anahtar kelime OVER() dır.

- **PARTITION BY:** Veriyi hangi gruplara ayıracağımızı belirler (GROUP BY gibi çalışır ama satırları birleştirmez).
 - **ORDER BY:** Pencere içindeki verilerin hangi sırayla işleneceğini belirler.
-

6.2. Sıralama Fonksiyonları (Ranking Functions)

Veri setindeki verileri belirli bir kritere göre numaralandırmak için kullanılır.

- **ROW_NUMBER():** Her satıra benzersiz bir sıra numarası verir.
- **RANK():** Aynı değerlere aynı sırayı verir ama bir sonraki numarayı atlar (1, 2, 2, 4 gibi).
- **DENSE_RANK():** Aynı değerlere aynı sırayı verir ama numara atlamaz (1, 2, 2, 3 gibi).

Dikey Kod Örneği (Top-N Analizi):

```
SELECT *
FROM (
    SELECT
        urun_adi,
        kategori,
        fiyat,
        -- Her kategoride en pahalı ürüne
        -- göre sıralama yapar (Numara atlamaz)
        DENSE_RANK() OVER (
            PARTITION BY kategori
            ORDER BY fiyat DESC
        ) AS sıra
    FROM urunler
) AS t
WHERE sıra <= 3;
```

6.3. Değer Kaydırma Fonksiyonları (Value Functions)

Zaman serisi analizlerinde "Bir önceki günün satışı neydi?" gibi soruları yanıtlar.

- **LAG():** Mevcut satırdan bir önceki satırın değerini getirir.
- **LEAD():** Mevcut satırdan bir sonraki satırın değerini getirir.

Dikey Kod Örneği (Büyüme Analizi):

```
SELECT
    tarih,
    toplam_satis,
    -- Bir önceki satırın (günün)
    -- satış verisini yan sütuna getirir
    LAG(toplam_satis) OVER (
        ORDER BY tarih
    ) AS onceki_gun_satis,
    -- Bugünkü satış ile dünkü satış
    -- arasındaki farkı hesaplar
    toplam_satis - LAG(toplam_satis) OVER (
        ORDER BY tarih
    ) AS gunluk_fark
FROM gunluk_rapor;
```

6.4. Agregasyon Pencere Fonksiyonları

Toplamları her satırın yanında göstererek kümülatif hesaplamalar yapmamızı sağlar.

- **Kümülatif Toplam (Running Total):** Başlangıçtan bugüne toplamın nasıl ilerlediğini görürüz.
- **Hareketli Ortalama (Moving Average):** Verideki dalgalanmaları yumuşatmak için kullanılır.

Dikey Kod Örneği:

```

SELECT
    tarih,
    satis,
    -- Tarih sırasına göre
    -- kümülatif (birikimli) toplam
    SUM(satis) OVER (
        ORDER BY tarih
    ) AS kumulatif_toplam,
    -- Mevcut satır ve önceki 6 satırı
    -- kapsayan (7 günlük) ortalama
    AVG(satis) OVER (
        ORDER BY tarih
        ROWS BETWEEN 6 PRECEDING
        AND CURRENT ROW
    ) AS yedi_gunluk_hareketli_ort
FROM satislar;

```

6.5. Veri Bilimciler İçin Kritik Kullanım Senaryoları

1. Yüzdelik Dilimler (NTILE)

Müşterileri harcamalarına göre 4 eşit gruba (Quartiles) ayırmak için kullanılır.

```

SELECT
    muster_i_id,
    -- Tüm müşterileri harcamalarına göre
    -- 4 farklı segmente ayırır (1-4 arası)
    NTILE(4) OVER (
        ORDER BY harcama_tutari DESC
    ) AS harcama_segmenti
FROM musteriler;

```

2. Duplicate (Tekrar Eden Veri) Temizleme

Bazı durumlarda DISTINCT yeterli olmaz. ROW_NUMBER() ile her tekrar eden kayda bir numara verip sadece "1" numaralı olanları seçerek temizlik yapılır.